

목차

1. 과제 1 설명 및 풀이
2. 과제 2 설명 및 풀이
3. 과제 3 설명 및 풀이

과제1

과제1 설명 :

과제1은 사용자가 입력할 원소의 개수를 입력하고 그 개수 만큼 정수형 원소를 입력받는다. 그리고 입력받은 정수형 데이터로 최댓값, 최솟값, 전체합, 평균을 구하는 문제이다.

풀이 :

이 과제를 풀이할 때 먼저 생각해야하는 것은 입력받은 정수형 데이터들을 저장할 배열을 만들어야 하는 것이다. 과제 조건에서 동적 할당을 이용해 배열로 풀이를 해야한다는 조건이 있다. 그래서 C언어에서는 malloc을 사용해 배열을 동적할당하여 사용했지만 C++에서는 malloc 대신 malloc과 비슷한 배열의 크기를 조절할 수 있는 vector를 사용했다. vector 또한 malloc과 비슷하게 내부적으로 메모리를 사용하는 자료구조이기 때문에 동적할당을 사용해야한다는 조건을 만족한다. vector를 사용할 때 <vector> 헤더를 사용했다. 코드에서는 arraySize() 함수를 사용해서 배열의 메모리(크기)를 입력할 원소 개수 만큼 할당하도록 했다. 그 후 배열에 main() 문에서 반복문을 사용해서 배열에 정수형 데이터를 할당했고 min_max(), sum_make(), avg_make() 함수를 사용해서 최대 최소 전체 합 평균을 구했다. sum_make() 와 avg_make() 함수에는 특별한 코드는 없다. 하지만 min_max() 함수에서 최대 최소값을 구할 때 C언어에서는 반복문과 조건문을 사용해 값을 구했다면 C++에서는 <algorithm> 헤더에 있는 min(), max() 함수를 사용했다. 이 함수들을 사용하면 함수 안에 비교할 인자들만 반복문을 사용해서 변경해준다면 스스로 비교를 통해 최대 최소 값을 구해준다. 그래서 이 코드에 적합하다고 판단해 사용하게 되었다.

결과 사진:

```
kdh@kdh:~/Desktop/21th_robit_intern_kimdohun/C++_QT_Day1$ ./HW1
몇 개의 원소를 할당하겠습니까? :10
정수형 데이터 입력:10
정수형 데이터 입력:20
정수형 데이터 입력:20
정수형 데이터 입력:10
정수형 데이터 입력:20
정수형 데이터 입력:30
정수형 데이터 입력:40
정수형 데이터 입력:20
정수형 데이터 입력:50
정수형 데이터 입력:20
최댓값: 50
최솟값: 10
전체합: 240
평균: 24.000000
kdh@kdh:~/Desktop/21th_robit_intern_kimdohun/C++_QT_Day1$
```

과제2

과제2 설명 :

과제2는 점의 개수를 입력하고 점의 개수만큼 점 좌표(X, Y)를 생성하고 좌표의 범위를 입력받는다. 범위는 X, Y 좌표 공통의 범위이다. 범위 제한을 고려한 점이 생성된다면 이 점들 사이의 거리를 구해 가장 큰 거리와 가장 작은 거리를 구하는 문제이다.

풀이 :

이 과제의 핵심적인 풀이는 구조체로 (x, y) 좌표를 구현하고 점의 개수만큼 구조체 배열을 생성한다. 생성한 구조체 배열 인자들을 사용해 점과 점 사이의 거리를 구하는 것이다. 먼저 입력 받을 점의 개수 만큼 구조체 배열을 생성해야한다. 그래서 과제 1과 같이 C++에서 사용가능한 vector를 사용했다. 과제2는 구조체 배열 크기를 main 함수에서 점의 개수를 입력 받은 후에 바로 할당할 수 있도록 점의 개수를 입력 받은 후 바로 크기를 할당했다. 그 다음에 좌표의 최대 최소 범위를 입력할 수 있도록 했다. 다음은 범위를 만족하는 수들 중에서 랜덤으로 (X, Y) 좌표를 생성해야한다. 랜덤으로 난수를 생성할 수 있도록 <random> 헤더 파일과 난수 생성 함수들을 사용해 범위에 만족하는 X, Y좌표를 생성하도록 했다. 점의 개수 만큼 반복문을 실행시켜 개수에 해당되는 점들을 모두 랜덤으로 생성했다. 그리고 두 점사이의 거리를 구하기 위해 따로 계산하는 함수 getDistance 함수를 만들어 모듈화를 진행했다. 이 함수에서는 제곱 계산, 루트 계산이 사용된다. 처음에는 제곱 계산을 <cmath> 헤더 파일의 pow 함수를 사용해 계산하려고 했지만 두 번 곱하는 것이 코드를 읽기 편하다고 생각해 제곱을 두 번 곱하는 것으로 표현했다. 그리고 루트는 표현할 방법이 없어 sqrt() 함수를 사용해 구현했다. 두 점사이의 거리는 정수를 떨어지지 않고 소수점이 나오기 쉬워 double 반환을 사용했다. analyze 함수에서 최댓값을 첫 번째 값으로 하고 순차적으로 비교하는 정렬 방식을 사용해 두 점 사이의 거리 중 최대 최소를 찾는 방식으로 코드를 구현했다.

결과 사진:

```
kdh@kdh:~/Desktop/21th_robot_intern_kimdohun/C++_QT_Day1$ ./HW2
점의 개수를 입력하세요:10
좌표의 범위 (최솟값)입력하세요:0
좌표의 범위 (최댓값) 입력하세요:20

랜덤 점 생성
점0: (3, 12)
점1: (16, 9)
점2: (9, 19)
점3: (10, 0)
점4: (13, 19)
점5: (3, 1)
점6: (8, 5)
점7: (1, 0)
점8: (4, 15)
점9: (16, 18)

===== 결과 =====
최대 거리 = 23.4307 점1 (1, 0), 점2(16, 18)
최소 거리 = 2.23607 점1 (3, 1), 점2(1, 0)
```

과제3

과제3 설명: 과제3은 복잡한 구조는 없지만 기능들이 많은 문제이다. 이 문제는 플레이어의 (X, Y) 좌표를 각각 1, -1씩 이동하면서 몬스터의 위치에 도달하고 도달하면 공격 명령을 내려 몬스터를 공격하여 몬스터의 체력을 감소시키면 종료되는 RPG가 게임을 구현하는 문제이다.

풀이

과제3을 풀이할 때 먼저 몬스터와 플레이어 클래스를 구현하고 main을 구현했다. 플레이어의 클래스에는 (X, Y) 이동 함수, 공격 함수, 플레이어의 최초 상태 정보를 저장하고 있는 함수, 게임 도중 플레이어 상태 정보를 알 수 있는 스테이스 함수를 구현했다. 몬스터는 이동할 필요가 없고 공격도 하지 않기 때문에 공격을 받고 HP를 감소시키는 함수, 몬스터의 최초 상태 정보를 저장하는 함수만 구현해주었다. 플레이어 클래스 선언부를 보면 플레이어와 관련된 모든 함수들에서 사용할 HP, MP, x, y를 변수를 선언해 줌으로써 포인터와 같은 참조 연산자를 사용할 필요가 없다. 몬스터 클래스도 마찬가지이다. 위의 네 변수들을 사용해 위치를 이동하고 MP를 감소시켜 MP가 0이 되거나 몬스터의 체력이 0이 되면 프로그램이 종료되도록 프로그램을 구현했다.

결과 사진:

```
게임 종료
kdh@kdh:~/Desktop/21th_robit_intern_kimdohun/C++_QT_Day1$ ./HW3
Type Command(A/U/D/R/L/S)
U
Y Position 1 moved!
Type Command(A/U/D/R/L/S)
U
Y Position 1 moved!
Type Command(A/U/D/R/L/S)
U
Y Position 1 moved!
Type Command(A/U/D/R/L/S)
U
Y Position 1 moved!
Type Command(A/U/D/R/L/S)
R
X Position 1 moved!
Type Command(A/U/D/R/L/S)
R
X Position 1 moved!
Type Command(A/U/D/R/L/S)
R
X Position 1 moved!
Type Command(A/U/D/R/L/S)
R
X Position 1 moved!
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:40
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:30
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:20
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:10
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:0
MONSTER DIED
```

어려웠던 점 및 추후 학습 계획

클래스와 객체에 대한 충분한 이해를 하지 못한 상태에서 코드를 작성하다 보니 클래스에 선언한 함수나 변수들을 활용할 때 어려움이 많았다. 어려움이 있을 때 마다 구글링이나 LLM을 통해 해결할 수 있었다. 추후에는 코드가 더 복잡해지고 클래스 개수가 많아질 것이라고 예상된다. 또한 이번 C++ 과제를 하면서 C언어 구조체 파트에 약하다는 것을 인지했다. 구조체 멤버 접근, 구조체 포인터, 구조체 배열등 이러한 개념 또한 class 못지 않게 중요한 개념이라고 판단했고 C++, QT 프로젝트 이전에는 이 두 개에 대한 깊은 이해와 활용

능력을 키울 것이다.